

Día 2: keyri

Requisitos previos:  Intermedio

Capítulo 5: Múltiples ventanas.....	1
Guardando referencias.....	2
Codependencia.....	3
Conectar las ventanas.....	3
Clase Login.....	3
Clase Dashboard.....	4
Métodos de ventana.....	4
Keyri, tu aplicación para contraseñas.....	4

¡Hola a tod@s!

En este tutorial, vamos a crear una aplicación utilizando *Python* + la librería *PySide6* y el constructor de UI *QT Designer*.

La idea principal de este aplicativo será conocer de alguna manera las bases de la **programación de eventos** construyendo un programa que nos permita generar en nuestro ordenador un archivo de texto con el cartón de un bingo.

Antes de continuar con el tutorial, hago un pequeño inciso para presentarme. Mi nombre es *Cristopher Méndez Cervantes* y actualmente me dedico a la programación de backend utilizando el framework de *Odoo* para crear aplicaciones funcionales de entorno ERP/CRM.

Llevo más de 2 años programando profesionalmente, siendo mi lenguaje natural Python, y actualmente me dedico en mi tiempo libre a compartir píldoras informativas, fomentando la creación de un ecosistema de programadores con amor por el conocimiento libre y accesible para todos.

¡Pero basta de presentaciones! Ha llegado el momento de emprender un viaje hacia un nuevo mundo de luz y color, ya que por fin pasaremos de scriptear en consola a generar nuestras propias interfaces.

Capítulo 5: Múltiples ventanas

Nuestra aplicación no suele ser una sola ventana. Muchas de las aplicaciones actuales tienen más de una ventana, y todas ellas conviven entre sí en un sistema perfecto donde cada una transfiere datos a otra en momentos clave.

La idea es que aprendamos a instanciar múltiples ventanas en QT, adoptando para ello las mejores técnicas.

Primero que nada, establezcamos la situación inicial:

Tenemos **3 ventanas**: main.py, login.py, y dashboard.py.

1. main se comunica con login, siendo la primera ventana que se abre al ejecutar el script.
2. login se comunica con dashboard.
3. dashboard se comunica con login de vuelta.

La pregunta del millón; ¿cómo podemos comunicar ventanas entre sí?

Guardando referencias

Primeramente deberemos situarnos en el fichero de login. A continuación, deberemos establecer las siguientes referencias:

```
21 from dashboard import Dashboard
22
23 class Login(object):
24     def setupUi(self, mainWindow):
25         #===== WINDOW CONFIG =====#
26
27         self.mainWindow = mainWindow
```

Como se puede observar, primero nos importaremos la clase Dashboard de nuestro fichero en la ruta actual de trabajo. Es muy importante, ya que cuando llegue el momento, nos instanciaremos la ventana como un objeto de esa clase.

Es buena práctica guardarse la referencia de la ventana actual, para ello, atendemos a la variable **mainWindow** que hace referencia al parámetro **mainWindow** del método **setupUi**.

¿Por qué hacemos esto? Porque esta es la configuración de ventana que tenemos que pasar a la otra ventana. A continuación, y justo debajo de esta variable, crearemos la ventana destino (dashboard).

```
self.mainWindow = mainWindow

#===== WINDOW 2 CONFIG =====#

self.window2 = QMainWindow()
self.ui2 = Dashboard()
self.ui2.setupUi(self.window2, self.mainWindow)
```

Con esto nos creamos un nuevo objeto de ventana **window2** y una instancia de Dashboard en **ui2**. La construcción de la ventana se realiza pasando como parámetro la referencia de la nueva ventana y la de origen que nos creamos antes (self.mainWindow).

Con esto, ya tenemos cubierta la creación de la ventana dentro de nuestra clase de Login, y lo que deberemos de atender a continuación son dos detalles adicionales.

Codependencia

```
21 class Dashboard(object):
22     def setupUi(self, vDos, mainWindow):
23         #===== context =====#
24
25         self.vDos = vDos
26         self.mainWindow = mainWindow
27
28         #=====#
29
```

En la clase de Dashboard tenemos que apuntar la referencia de la ventana de origen. Como podemos ver, es mainWindow, que si recordamos, nos la hemos pasado desde la clase Login.

Como se ha mencionado, a su vez, es buena práctica apuntar las referencias de ventana en variables.

Conectar las ventanas

Ha llegado la hora de la verdadera magia del asunto: conectar las ventanas mediante eventos de clic en botones. Crearemos dos slots; uno para la clase Login y otro para la clase Dashboard.

Clase Login

Nos crearemos un nuevo método que lo que haga sea instanciar la nueva ventana y la conectaremos de la siguiente manera.

```
#===== MAIN SLOTS =====#

self.btn_login.clicked.connect(self.instantiate_dashboard)
```

Solo queda crear el método con la siguiente información base:

```
self.window2.show()
self.mainWindow.hide()
self.ui2.load_current_user((self.lnEdit_usuario.text(), self.lnEdit_contrasenia.text()))
```

¡Nota! window2 es la instancia de ventana Dashboard. Podemos atender a su contenido mediante ui2, que es la instancia del objeto. Por eso yo llamo a un método desde la ventana de login.

Clase Dashboard

El mismo procedimiento, pero atendiendo a las propiedades de su clase guardadas que hacían referencia a la clase origen:

```
def logout(self):
    '''Logs out.'''
    self.vDos.close()
    self.mainWindow.lnEdit_usuario.clear()
    self.mainWindow.lnEdit_contrasenia.clear()
    self.mainWindow.show()
```

Métodos de ventana

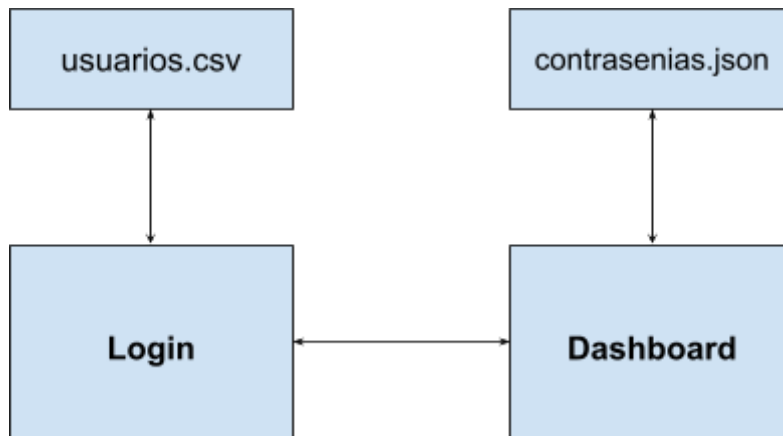
Los métodos close, show y hide son característicos en QT. Close solicita el cierre de un widget, liberando memoria. Show muestra un widget, y hide lo oculta.

Keyri, tu aplicación para contraseñas

En su diseño, Keyri trata de ser una aplicación multiventana con su propio inicio/cierre de sesión (por tanto, requiere de autenticación del usuario).

Digamos que necesitamos una aplicación que nos permita guardar de una manera rápida y sencilla nuestras contraseñas, ya que tenemos muchas cuentas y nos podemos olvidar de alguna.

Para ello, atenderemos al siguiente esquema de la aplicación:



Tal y como se puede apreciar, existen dos ficheros con diferentes tipos de datos que cargaremos en las diferentes clases principales del programa:

1. **usuarios.csv**: Contiene la información clave con **nombre de usuario** y **contraseña** de cada usuario del sistema.
2. **contrasenias.json**: Contiene para cada nombre de usuario autenticado diferentes cuentas asociadas y su información privilegiada. Los campos que se encuentran en este fichero son:
 - a. **servicio**: El servicio o aplicativo del cual proviene su contraseña. Ejemplo: Facebook.
 - b. **usuario_cuenta**: El nombre de cuenta o usuario para el servicio.
 - c. **contraseña**: La contraseña asociada a la cuenta.

Los requisitos funcionales de esta primera versión son los siguientes:

1. El archivo main da paso al login, cargando todos los usuarios del archivo csv en "segundo plano". Cuando un usuario intenta acceder al dashboard, deberá estar previamente inscrito en los diferentes archivos csv y json con todos sus datos. Nota: Esta versión es totalmente académica y experimental. En entornos reales un login tiene muchos más campos y opción a creación de nuevas cuentas. Por ahora, usaremos las cuentas que vienen dadas por el fichero del repositorio.
2. El archivo dashboard recibe los datos del usuario como parámetro y a continuación busca en el documento json sus datos asociados. Utiliza métodos de su propia clase.
3. El dashboard renderiza en la tabla los datos del usuario. Para ello, utiliza los métodos `setRowCount()`, `setItem()` e `insertRow()`, iterando sobre tuplas.
4. El usuario puede hacer logout para acceder a otra cuenta y cargar sus propios datos personales.
5. El sistema contempla validaciones básicas en el login y en el dashboard.